Examen MULWEB3 UT1

Enunciado

Al cargar la página debe pedirse al usuario, mediante `prompt()`, **una lista de números separados por comas** (ej. `1,2,3,1,1,3`). El programa debe calcular cuántos **pares buenos** hay. Un par `(i, j)` es bueno si `i < j` y `nums[i] === nums[j]`.

Requisitos técnicos

1. Implementar la función:
   - function name: countGoodPairs(nums)
   - `nums` es un `Array` de números.
   - Devuelve un `Number` con la cantidad de pares buenos.
   - Complejidad ideal: O(n) usando un objeto/mapa para contar repeticiones.
2. Entrada:
   - Pedir la lista con `prompt()` al cargar la página.
   - Convertir los elementos a enteros (ignorar espacios alrededor).
   - Si la entrada está vacía o contiene elementos no numéricos, **no** usar `alert()`: mostrar un mensaje de error en la página (mediante creación de elementos DOM).
3. Salida:
   - Mostrar en la página (DOM) al menos:
     - Un título o encabezado (`<h2>`): “Resultado”.
     - Un párrafo con la **lista** interpretada.
     - Un párrafo con el **número de pares buenos**.
   - Crear los nodos DOM. No utilizar `innerHTML` para inyectar todo de golpe (sí puedes para texto pequeño).

Comportamiento esperado / ejemplos

- Entrada (prompt): `1,2,3,1,1,3`
  - Mostrar: `Lista: [1,2,3,1,1,3]`
  - Mostrar: `Número de pares buenos: 4`
- Entrada: `1,1,1,1` → `Número de pares buenos: 6`
- Entrada inválida: `1,2,a,3` → Mostrar en la página un mensaje de error claro (“Entrada inválida: 'a' no es un número”).

Criterios de corrección

- Funcionalidad correcta de `countGoodPairs` y resultado en página: **60%**
- Validación de entrada y mensajes claros en DOM (sin `alert`): **20%**
- Uso correcto de creación e inserción de elementos DOM: **15%**
- Código legible, use stricto, comentado:**5%**

Entregable

Archivos a entregar, un zip con: `main.js,main.html`